PONTIFÍCIA UNIVERSIDADE CATÓLICA

Engenharia de Software • Testes e Qualidade de Software

Detecção de Bad Smells e Refatoração Segura

Análise estática de código com ESLint e SonarJS, identificação de débitos técnicos e refatoração segura apoiada por suíte de testes automatizados.

Disciplina: Qualidade e Testes de Software

Trabalho: Detecção de Bad Smells e Refatoração Segura

Aluno: Luis Henrique Gonçalves Barbosa

Matrícula: Aluno PUC Minas

Repositório: `https://github.com/BGLuis/bad-smells-js-refactoring`

Belo Horizonte • 2026

1. ANÁLISE MANUAL DE BAD SMELLS (CAÇA AOS SMELLS)

A inspeção estática manual do arquivo `src/ReportGenerator.js` revelou diversos sintomas clássicos de degradação de design estrutural de software catalogados por Martin Fowler. Foram identificados três Bad Smells críticos com impacto direto na testabilidade e manutenibilidade do sistema:

Smell 1: Long Method (Método Longo) e Violação de SRP

O método `generateReport` concentra em uma única função de 69 linhas todas as responsabilidades do domínio: determinação do formato de saída (CSV ou HTML), cabeçalhos, filtragem e autorização de itens por perfil de usuário (ADMIN vs USER), mutação direta de propriedades (`item.priority`), cálculo aritmético de totais e renderização de rodapé.

Impacto na manutenção e testes: Essa concentração impede a reutilização de código e obriga qualquer nova alteração (ex: adicionar suporte a JSON) a tocar no fluxo de dados de todos os outros formatos. Para testes, gera matrizes combinatórias exponenciais e impede testar a lógica de agregação ou filtragem de forma unitária isolada.

Smell 2: Alta Complexidade Cognitiva e Ciclomática (Deeply Nested Logic)

O laço de iteração dos itens possui aninhamento de 4 níveis de estruturas de decisão sequenciais: `for -> if (user.role === 'ADMIN') -> if (item.value > 1000) -> if (reportType === 'CSV')`. Conforme definido pelo SonarSource, a complexidade cognitiva mede a sobrecarga mental necessária para um desenvolvedor rastrear o fluxo de execução. A função obteve complexidade 27, ultrapassando amplamente o limite padrão (5 a 15).

Impacto na manutenção e testes: Dificulta a predição visual de fluxos de exceção e ramos não executados, multiplicando a probabilidade de regressões durante correções de bugs. A cobertura completa de todos os ramos (branch coverage) torna-se altamente suscetível a pontos cegos.

Smell 3: Duplicated Code (Código Duplicado) e Falta de Abstração

A formatação das linhas de CSV (`${item.id},${item.name},${item.value},${user.name}\n`) e a operação de soma `total += item.value;` aparecem replicadas linha por linha nos ramos de `ADMIN` e `USER`. Além disso, a condicional que chaveia o formato `reportType` se repete em 3 pontos distintos do método (cabeçalho, corpo e rodapé).

Impacto na manutenção e testes: Viola o princípio fundamental *Don't Repeat Yourself (DRY)*. Se o formato de uma coluna mudar ou houver ajuste no arredondamento do valor, o desenvolvedor precisa lembrar de replicar a mudança em pontos espalhados, correndo o risco de atualizar apenas um ramo e introduzir inconsistência em produção.

2. RELATÓRIO DA FERRAMENTA DE DETECÇÃO AUTOMÁTICA (ESLINT + SONARJS)

Para comprovar os achados da análise manual por meios automatizados, executou-se a ferramenta de análise estática **ESLint** com o plugin **eslint-plugin-sonarjs** configurado conforme as diretrizes do trabalho prático.

```
$ npx eslint src/

/home/luis/Documents/puc/testes/bad-smell/src/ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one                                          sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

Papel do eslint-plugin-sonarjs: Enquanto linters tradicionais apenas apontam erros sintáticos (ponto e vírgula, nomes de variáveis não utilizados), o SonarJS introduz métricas cognitivas e de design estrutural. Ele comprovou objetivamente que a complexidade cognitiva do método era de **27** (mais de cinco vezes superior ao teto seguro estabelecido), indicando com precisão matemática os locais exatos onde a legibilidade humana colapsa.

3. PROCESSO DE REFATORAÇÃO SEGURA

O Bad Smell mais crítico selecionado para refatoração foi a **Alta Complexidade Cognitiva combinada ao Acoplamento de Responsabilidades de Formatação**. Para sanar esse problema sem comprometer a integridade do sistema, aplicou-se um conjunto coordenado de técnicas fundamentadas no catálogo de Martin Fowler:

- **Replace Conditional with Strategy / Polymorphism:** As variações de cabeçalho, linha e rodapé dos formatos CSV e HTML foram extraídas para classes formatadoras dedicadas (`CsvReportFormatter` e `HtmlReportFormatter`). Com isso, eliminou-se o cascateamento de verificações condicionais e abriu-se o sistema para extensão (Open/Closed Principle).
- **Extract Method (Extração de Método):** O processamento foi segmentado em responsabilidades atômicas: `_resolveFormatter` , `_filterAndProcessItems` , `_calculateTotal` e `_renderReport` .
- **Decompose Conditional & Imutabilidade:** As condições de filtragem e atribuição de prioridade foram isoladas com métodos funcionais (`filter` , `map` , `reduce`), eliminando a mutação in-place perigosa dos parâmetros de entrada.
- **Replace Magic Numbers with Named Constants:** Literais mágicos como `1000` e `500` foram formalizados em `THRESHOLDS.ADMIN_PRIORITY` e `THRESHOLDS.USER_MAX_VALUE` .

Comparativo de Código: Antes vs. Depois

Antes (Código Acoplado e Aninhado - Complexidade 27)

```
// src/ReportGenerator.js
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) item.priority = true;
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? 'style="font-weight:bold;"' : '';
      report += `<tr ${style}><td>${item.id}...`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') { ... }
      else if (reportType === 'HTML') { ... }
    }
  }
}
```

Depois (Estratégia Limpa e Modular - Complexidade ≤ 2)

```
// src/ReportGenerator.refactored.js
generateReport(reportType, user, items) {
  const formatter = this._resolveFormatter(reportType);
  const visibleItems = this._filterAndProcessItems(user, items);
  const total = this._calculateTotal(visibleItems);

  return this._renderReport(formatter, user, visibleItems, total);
}

_filterAndProcessItems(user, items) {
  if (user.role === ROLES.ADMIN) {
    return items.map(item => ({
      ...item,
      priority: item.value > THRESHOLDS.ADMIN_PRIORITY
    }));
  }
  if (user.role === ROLES.USER) {
    return items.filter(item => item.value <= THRESHOLDS.USER_MAX_VALUE);
  }
  return [];
}
```

Validação Final pelo Linter (Zero Smells)

A execução do ESLint sobre o arquivo refatorado demonstrou a eliminação integral de todos os problemas relatados:

```
$ npx eslint src/ReportGenerator.refactored.js
✔ 0 problems (0 errors, 0 warnings)
```

4. USO DA SUÍTE DE TESTES COMO REDE DE SEGURANÇA

A refatoração só pode ser considerada segura se garantir de forma incontestável a **preservação de comportamento observável**. Para atingir essa certeza, os testes automatizados com Jest foram executados a cada transformação de código.

```
$ npm test

PASS __tests__/ReportGenerator.refactored.test.js
PASS tests/ReportGenerator.refactored.test.js
PASS __tests__/ReportGenerator.test.js
PASS tests/ReportGenerator.test.js

Test Suites: 4 passed, 4 total
Tests:       20 passed, 20 total
Snapshots:   0 total
Time:        0.251 s
```

Métrica de Qualidade	Código Original (ReportGenerator.js)	Código Refatorado (ReportGenerator.refactored.js)
Complexidade Cognitiva	27 (Violação: limite 15)	≤ 2 por método (100% em conformidade)
Erros de Linter (ESLint/SonarJS)	2 erros bloqueantes	0 erros, 0 advertências
Aderência ao Princípio SRP	Baixa (1 classe acumulando 5 papéis)	Alta (Formatadores isolados + Gerador coordenador)
Preservação de Testes (Segurança)	Original com inconsistência em <tr >	100% de sucesso em todos os cenários (Admin, User, Vazio)

5. CONCLUSÃO

O presente trabalho demonstrou na prática o ciclo virtuoso da engenharia de software moderna: **detectar objetivamente com ferramentas automatizadas, compreender as causas-raiz via catálogo de padrões e refatorar sob a custódia inegociável de uma suíte de testes automatizados**.

A suíte de testes atuou como uma verdadeira *rede de segurança*: a cada pequena alteração estrutural (como a transição do laço iterativo imperativo para o padrão Strategy com formatadores polimórficos), o feedback instantâneo do Jest assegurava que nenhuma regressão ou quebra de contrato foi introduzida.

A eliminação dos Bad Smells resultou em um código legível, testável e manutenível. A complexidade cognitiva desabou de 27 para valores unitários inferiores a 2, capacitando a equipe a evoluir o sistema no futuro — por exemplo, adicionando novos formatos de relatório ou novos perfis de acesso — sem a necessidade de reescrever ou colocar em risco a lógica existente.